

반려로봇 my petbot 만들기



Contents

1 주제 : 우리가 만들려는 것은

1.1 작고 귀여운 반려 로봇 입니다.

1.2 나만의 반려로봇 만들기

1.3 프로젝트 개요

1.4 거창하진 않지만 - 감정 기반 대화 시스템이란

1.5 개념

1.6 우리가 다룰 감정 6개

1.7 감정 대응 전략

2 구현 과정을 코드로

2.1 기초 코드

2.1.1 필요한 아이들 import

2.1.2 led, lcd, 모터 초기화

2.2 led에 색상으로 감정 표현

2.2.1 필요한 감정, 색상 define

2.2.2 몇몇 색상을 순환해서 보여주는 함수

2.2.3 실행해보자

2.2.4 어떤 python 클래스가 지원하는 함수들을 확인하고 싶다면

2.2.5 clear 시키면 이렇게

2.2.6 이제 감정에 따라 led를 기동하는 함수

2.3 lcd에 감정을 표현하는 표정

2.3.1 lcd에 감정을 표현할 수 있는 그림파일의 종류 확인하기

2.3.2 lcd에 gif 보기

2.3.3 실행하면 이렇게

2.3.4 그걸 함수로, 재생시간이 조금 길어서 줄이는걸로

2.4 모터의 움직임으로 감정 표현

2.4.1 좌우로 흔들흔들 하는 코드

2.4.2 앞뒤로 흔들

2.4.3 앞으로 한 번 움찔

2.4.4 이제 감정에 따라 움직임 정의

2.4.5 하나만 테스트

2.4.6 angry한 행동

2.4.7 interest한 행동

2.5 카메라 이미지 가져오기

2.5.1 camera frame 하나만 가져옴

2.5.2 camera 열고, frame 하나 가져오기

2.5.3 카메라 테스트

3 이제 본게임 - 지능(AI)과 연결

3.1 LLM 연결

3.1.1 OpenAI API 연결

3.1.2 반듯이 감정을 infer 추론하라

3.1.3 text와 hist를 받아 지금의 답변을 내 놓기

3.1.4 메시지를 api로 gpt에 던지고 답변을 받는 함수

3.1.5 이제 채팅을 통해 감정표현을 하면서 대화하는 함수

3.1.6 대화시작

3.1.7 그게 이 결과

4 이번에는 사진을 보고 감정 표현

4.1 사진을 보고 감정을 읽으라는 프롬프트

4.2 base64를 이용해서 이미지를 행렬로 변환

4.3 OpenAI API에게 이미지 보내고 응답 받기

4.4 감정 값 처리

4.5 이미지 던지고, 응답 받아서, 감정값 추출

4.6 이미지 캡처하고 감정 받기

4.7 전체과정

4.8 시작~

반려로봇 my petbot 만들기

Exported from Confluence · <https://pinkwink.atlassian.net>

주제 : 우리가 만들려는 것은

작고 귀여운 반려 로봇 입니다.



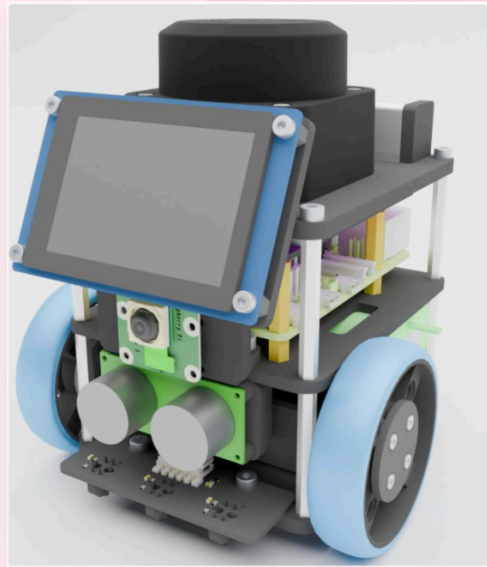
나만의 반려로봇 만들기

나만의 반려로봇 만들기

감정 기반 대화 시스템으로 배우는 반려로봇 구축

제작 : 민형기 (핑크랩)

2025.11.15



프로젝트 개요

프로젝트 개요

- 🎯 **목표:** 감정(emotion)에 따라 자연스럽게 반응하는 반려로봇 대화 시스템 이해
- 🔑 **핵심:** 6가지 감정 태그(hello, happy, sad, angry, interest, fun)에 따른 응답 패턴
- 📄 **자료 특성:** 코드 구현 대신 예시 대화 데이터(assistant_message + emotion) 중심
- 📈 **기대효과:** 공감형 대화, 친밀감 형성, 교육·치료·일상 보조 활용

거창하진 않지만 - 감정 기반 대화 시스템이란

감정 기반 대화 시스템이란?

-  **개념:** 사용자의 입력 또는 맥락에 감정 태그를 부여하고, 감정에 맞는 응답을 생성하는 방식
-  **구현방식:** 단순 규칙(Rule-based)부터 ML/NLP까지 다양한 접근 가능
-  **장점 1:** 더 따뜻하고 인간적인 상호작용으로 사용자 경험 향상
-  **장점 2:** 사용자 몰입도와 지속적 사용 의지 증가
-  **장점 3:** 맥락과 상황에 맞는 자연스러운 대화 흐름 제공

개념



입력 계층

사용자의 텍스트 입력을 처리합니다. 향후 음성/영상 입력으로 확장 가능합니다.

감정 태깅

입력 텍스트를 분석하여 hello/happy/sad/angry/interest/fun 중 하나로 분류합니다.

응답 엔진

감정 태그에 맞는 응답을 선택하거나 생성합니다. 룰 기반 또는 모델 기반으로 구현 가능합니다.

대화 관리

대화의 맥락을 유지하고, 세션 상태를 관리하며 안전성 필터를 적용합니다.

우리가 다룰 감정 6개

6가지 감정 상태



hello

인사/시작 맥락. 친근한 톤으로 대화 시작



happy

기쁜 감정. 긍정·칭찬·축하 중심 응답



sad

슬픈 감정. 위로·공감·격려 표현



angry

화남. 진정·사과·문제 해결 의지 강조



interest

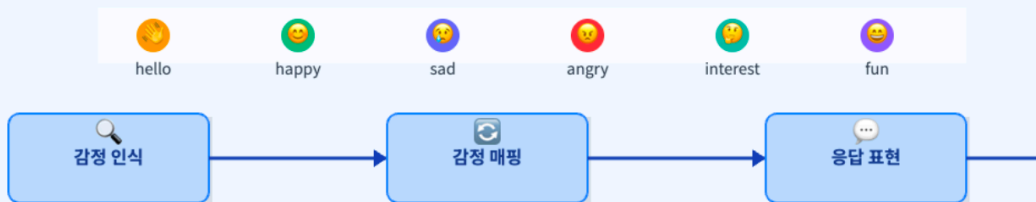
관심/호기심. 질문·탐색·추천 제안



fun

장난/유머. 가벼운 농담과 즐거운 톤

감정 대응 전략



인식 (감정 결정)

사용자 입력을 분석하여 6가지 감정 중 하나로 분류합니다. 규칙 기반 방식이나 머신러닝 분류 모델을 활용할 수 있습니다.

매핑 (전략 선택)

결정된 감정에 따라 적합한 응답 템플릿과 전략을 선택합니다. 각 감정으로 고유한 응답 패턴이 적용됩니다.

표현 (스타일 조정)

선택된 전략에 따라 문체, 이모지, 어휘 강도를 조절합니다. 예를 들어 happy 감정은 밝고 활기찬 표현을, sad는 공감적 어조를 사용합니다.

안전장치 (부정 감정 처리)

부정적 감정(sad, angry)을 처리할 때 유해 표현을 완화하고 중립적 대안이나 해결책을 제안하는 안전 필터를 적용합니다.

구현 과정을 코드로

기초 코드

필요한 아이들 import

```
from pinky_lcd.pinky_lcd import LCD
from pinkylib import LED, Camera, Motor
from PIL import Image, ImageSequence
from IPython.display import display
import time, io, base64, json
```

Python

led, lcd, 모터 초기화

```
led = LED()
lcd = LCD()
motor = Motor(); motor.enable_motor()
```

Python

모터 #1를 성공적으로 '속도 제어 모드'로 설정했습니다.
모터 #2를 성공적으로 '속도 제어 모드'로 설정했습니다.

led에 색상으로 감정 표현

필요한 감정, 색상 define

```
EMOTIONS = ['angry', 'basic', 'bored', 'fun', 'happy', 'hello', 'interest', 'sad']
```

Python

```
RED=(255,0,0); ORANGE=(255,127,0); YELLOW=(255,255,0)
GREEN=(0,255,0); BLUE=(0,0,255); INDIGO=(75,0,130)
VIOLET=(148,0,211); WHITE=(255,255,255)
```

```
RAINBOW = [RED,ORANGE,YELLOW,GREEN,BLUE,INDIGO,VIOLET]
```

Python

몇몇 색상을 순환해서 보여주는 함수

```
def play_rainbow_once(dt=0.1):
    for c in RAINBOW:
        led.fill(c); time.sleep(dt)
```

✓ 0.0s

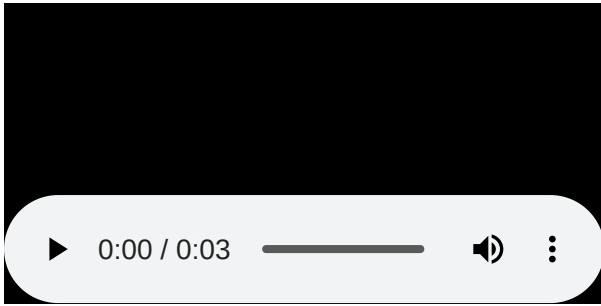
Python

실행해보자

```
play_rainbow_once(0.15)
```

✓ 1.0s

Python



어떤 python 클래스가 지원하는 함수들을 확인하고 싶다면

```
led.__dir__()
```

✓ 0.0s

Python

```
['_strip',  
'count',  
'__module__',  
'__init__',  
'__enter__',  
'__exit__',  
'show',  
'clear',  
'set_pixel',
```

clear 시키면 이렇게

```
led.clear()
```

✓ 0.0s

Python

이제 감정에 따라 led를 기동하는 함수

```
def play_led_emotion(e):  
    if e=="angry": led.fill(RED)  
    elif e=="happy": led.fill(YELLOW)  
    elif e=="fun": play_rainbow_once(0.08)  
    elif e=="sad": led.fill(BLUE)  
    elif e=="bored": led.fill(INDIGO)  
    elif e=="interest": led.fill(GREEN)  
    elif e=="hello":  
        led.fill(WHITE); time.sleep(0.2)  
        led.fill((0,0,0)); time.sleep(0.1); led.fill(WHITE)  
    else: led.fill(WHITE)
```

✓ 0.0s

Python

lcd에 감정을 표현하는 표정

lcd에 감정을 표현할 수 있는 그림파일의 종류 확인하기

```
import os

emotion_folder_path = '/home/pinky/pinky_pro/src/pinky_pro/pinky_emotion/emotion/'
files = os.listdir(emotion_folder_path)

print(files)
```

✓ 0.0s

Python

['angry.gif', 'basic.gif', 'bored.gif', 'hello.gif', 'interest.gif', 'fun.gif', 'happy.gif', 'sad.gif']

lcd에 gif 보기

```
print("Start GIF")

gif = Image.open(emotion_folder_path + "fun.gif")
for frame in ImageSequence.Iterator(gif):
    lcd.img_show(frame)

lcd.clear()

print("End GIF")
```

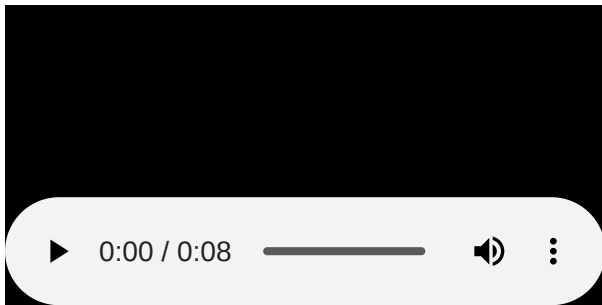
✓ 7.1s

Python

Start GIF

End GIF

실행하면 이렇게



그걸 함수로, 재생시간이 조금 길어서 줄이는걸로

```
def play_emotion(emotion):
    emotion_folder_path = '/home/pinky/pinky_pro/src/pinky_pro/pinky_emotion/emotion/'
    emotion_path = f"{emotion_folder_path}/{emotion}.gif"
    gif = Image.open(emotion_path)

    n = 0
    for frame in ImageSequence.Iterator(gif):
        lcd.img_show(frame)
        n += 1
        if n > 20:
            break

    print("End Emotion")
```

✓ 0.0s

Python

모터의 움직임으로 감정 표현

좌우로 흔들흔들 하는 코드

```
def motor_wiggle_lr(c=3, sp=20, dt=0.3):
    for _ in range(c):
        motor.move(sp, -sp); time.sleep(dt)
        motor.move(-sp, sp); time.sleep(dt)
    motor.stop()
```

✓ 0.0s

Python

앞뒤로 흔들

```
def motor_wiggle_fb(c=2, sp=5, dt=0.1):
    for _ in range(c):
        motor.move(sp, sp); time.sleep(dt)
        motor.move(-sp, -sp); time.sleep(dt)
    motor.stop()
```

✓ 0.0s

Python

앞으로 한 번 움찔

```
def motor_nod_forward(sp=10, dt=0.1):
    motor.move(sp, sp); time.sleep(dt); motor.stop()
```

✓ 0.0s

Python

이제 감정에 따라 움직임 정의

```
def play_motor_emotion(e):
    if e in ["happy", "fun", "hello"]:
        motor_wiggle_lr()
    elif e=="sad":
        motor_wiggle_fb()
    elif e=="angry":
        motor_wiggle_fb(c=1, sp=70, dt=0.2)
    elif e=="interest":
        motor_nod_forward(sp=35, dt=0.25)
    elif e=="bored":
        motor_wiggle_lr(c=1, sp=20, dt=0.5)
    else:
        motor.stop()
```

✓ 0.0s

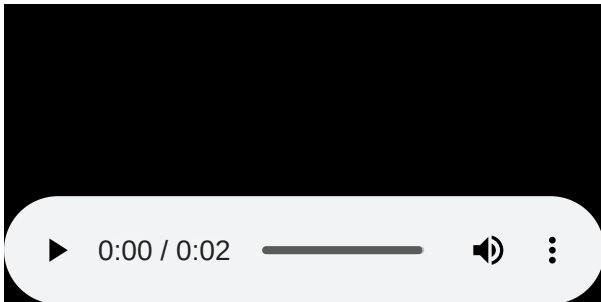
Python

하나만 테스트

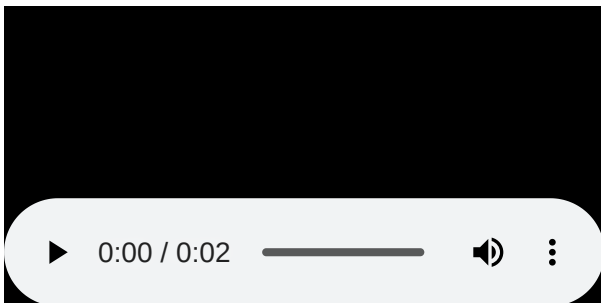
```
play_motor_emotion("angry")
```

Python

angry한 행동



interest한 행동



카메라 이미지 가져오기

camera frame 하나만 가져옴

```
def capture_one_frame_bytes(cam):
    frame = cam.get_frame() # 환경에 맞게 메서드 이름 조정
    img = Image.fromarray(frame[:, :, ::-1]) # BGR -> RGB
    buf = io.BytesIO(); img.save(buf, format="JPEG")
    return buf.getvalue()
```

✓ 0.0s

Python

camera 열고, frame 하나 가져오기

```
def test_camera_once():
    cam = Camera(); cam.start()
    img_bytes = capture_one_frame_bytes(cam)
    display(Image.open(io.BytesIO(img_bytes)))
    cam.close()
```

✓ 0.0s

Python

카메라 테스트

test_camera_once()

✓ 0.6s

Python

```
[2:32:50.493834811] [6232] INFO Camera camera_manager.cpp:325 libcamera v0.3.2+99-1230f78d
[2:32:50.520130003] [11480] INFO RPI pisp.cpp:695 libpisp version v1.0.7 28196ed6edcf 28-11-2024 (14:00:1
[2:32:50.533831596] [11480] INFO RPI pisp.cpp:1154 Registered camera /base/axi/pcie@120000/rp1/i2c@80000/
[2:32:50.545220815] [6232] WARN V4L2 v4l2_pixelformat.cpp:346 Unsupported V4L2 pixel format RPBP
[2:32:50.546184734] [6232] INFO Camera camera.cpp:1197 configuring streams: (0) 640x480-RGB888 (1) 640x48
[2:32:50.546367918] [11480] INFO RPI pisp.cpp:1450 Sensor: /base/axi/pcie@120000/rp1/i2c@80000/ov5647@36
```



이제 본게임 - 지능(AI)과 연결

LLM 연결

OpenAI API 연결

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

✓ 0.9s

Python

반듯이 감정을 infer 추론하라

```
SYSTEM_PROMPT = f"""
You are a helpful robot friend.

Always reply ONLY in JSON:
{{
  "assistant_message": "<reply in Korean>",
  "emotion": "<one of: {'', '.join(EMOTIONS)}>"
}}

Infer the user's current emotion.
If unclear, use "basic".
No extra fields, no text outside JSON.
"""
```

Python

text와 hist를 받아 지금의 답변을 내 놓기

```
def build_chat_messages(text, hist):
    msgs = [{"role": "system", "content": SYSTEM_PROMPT}]
    msgs += hist
    msgs.append({"role": "user", "content": text})
    return msgs
```

Python

메시지를 api로 gpt에 던지고 답변을 받는 함수

```
def request_chat_emotion(msgs, model="gpt-4.1-mini"):
    res = client.chat.completions.create(
        model=model, messages=msgs, temperature=0.4
    )
    raw = res.choices[0].message.content
    print("RAW:", raw)
    try:
        data = json.loads(raw)
        msg = data.get("assistant_message", "")
        emo = data.get("emotion", "basic")
    except json.JSONDecodeError:
        msg, emo = raw, "basic"
    if emo not in EMOTIONS: emo = "basic"
    return msg, emo, raw
```

✓ 0.0s

Python

이제 채팅을 통해 감정표현을 하면서 대화하는 함수

```
def chat_with_emotion(text, history=None):  
    if history is None: history = []  
    msgs = build_chat_messages(text, history)  
    reply, emo, raw = request_chat_emotion(msgs)  
    history += [{"role": "user", "content": text},  
               {"role": "assistant", "content": raw}]  
    return reply, emo, history
```

✓ 0.0s

Python

대화시작

```
history = []  
print("대화 시작 (종료: quit/exit/종료)")  
  
while True:  
    user = input("You: ")  
    if user.lower() in ["quit", "exit", "종료"]:  
        break  
    reply, emo, history = chat_with_emotion(user, history)  
    print(f"REQ : {user}")  
    print(f"Bot({emo}):", reply)  
    play_led_emotion(emo)  
    play_emotion(emo)  
    play_motor_emotion(emo)
```

Python

그게 이 결과



이번에는 사진을 보고 감정 표현

사진을 보고 감정을 읽으라는 프롬프트

```
VISION_SYSTEM_PROMPT = f"""
You see a single face image.

Return ONLY JSON:
{{ "emotion": "<one of: {'', '.join(EMOTIONS)}>" }}

Choose the best matching emotion from the face.
If unclear, use "basic".
"""
```

Python

base64를 이용해서 이미지를 행렬로 변환

```
def encode_image_bytes(img_bytes: bytes) -> str:
    return base64.b64encode(img_bytes).decode("utf-8")
```

Python

- OpenAI Vision API에 이미지를 보내려면
"이미지를 문자열 형태(Base64)로 변환해서" 보내야 한다.
- 바이트(raw binary)는 JSON이나 HTTP로 직접 넣을 수 없다.

- 그래서 **base64**라는 인코딩 방식으로
→ 텍스트로 안전하게 변환해야 한다.

OpenAI API에게 이미지 보내고 응답 받기

```
def request_vision_raw_response(b64_img, model="gpt-4.1-mini"):
    msg = [
        {"role": "system", "content": VISION_SYSTEM_PROMPT},
        {"role": "user", "content": [
            {"type": "text", "text": "Choose ONE emotion."},
            {"type": "image_url",
             "image_url": {"url": f"data:image/jpeg;base64,{b64_img}"}}
        ]}
    ]
    res = client.chat.completions.create(
        model=model, messages=msg, temperature=0, max_completion_tokens=64
    )
    raw = res.choices[0].message.content
    print("RAW:", raw)
    return raw
```

Python

감정 값 처리

```
def parse_vision_emotion(raw: str) -> str:
    try:
        data = json.loads(raw)
        emo = data.get("emotion", "basic").strip()
    except json.JSONDecodeError:
        emo = "basic"
    if emo not in EMOTIONS: emo = "basic"
    return emo
```

Python

이미지 던지고, 응답 받아서, 감정값 추출

```
def detect_emotion_from_image_bytes(img_bytes: bytes) -> str:
    b64 = encode_image_bytes(img_bytes)
    raw = request_vision_raw_response(b64)
    return parse_vision_emotion(raw)
```

Python

이미지 캡처하고 감정 받기

```
def capture_image(cam):
    img_bytes = capture_one_frame_bytes(cam)
    display(Image.open(io.BytesIO(img_bytes)))
    return img_bytes

def process_emotion(img_bytes):
    emo = detect_emotion_from_image_bytes(img_bytes)
    print("Detected emotion:", emo)
    return emo
```

Python

전체과정

```
def run_camera_emotion_once():
    cam = Camera(); cam.start()
    img_bytes = capture_image(cam)
    emo = process_emotion(img_bytes)
    play_led_emotion(emo)
    play_emotion(emo)
    play_motor_emotion(emo)
    cam.close()
```

Python

시작~

```
run_camera_emotion_once()
```

Python



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio